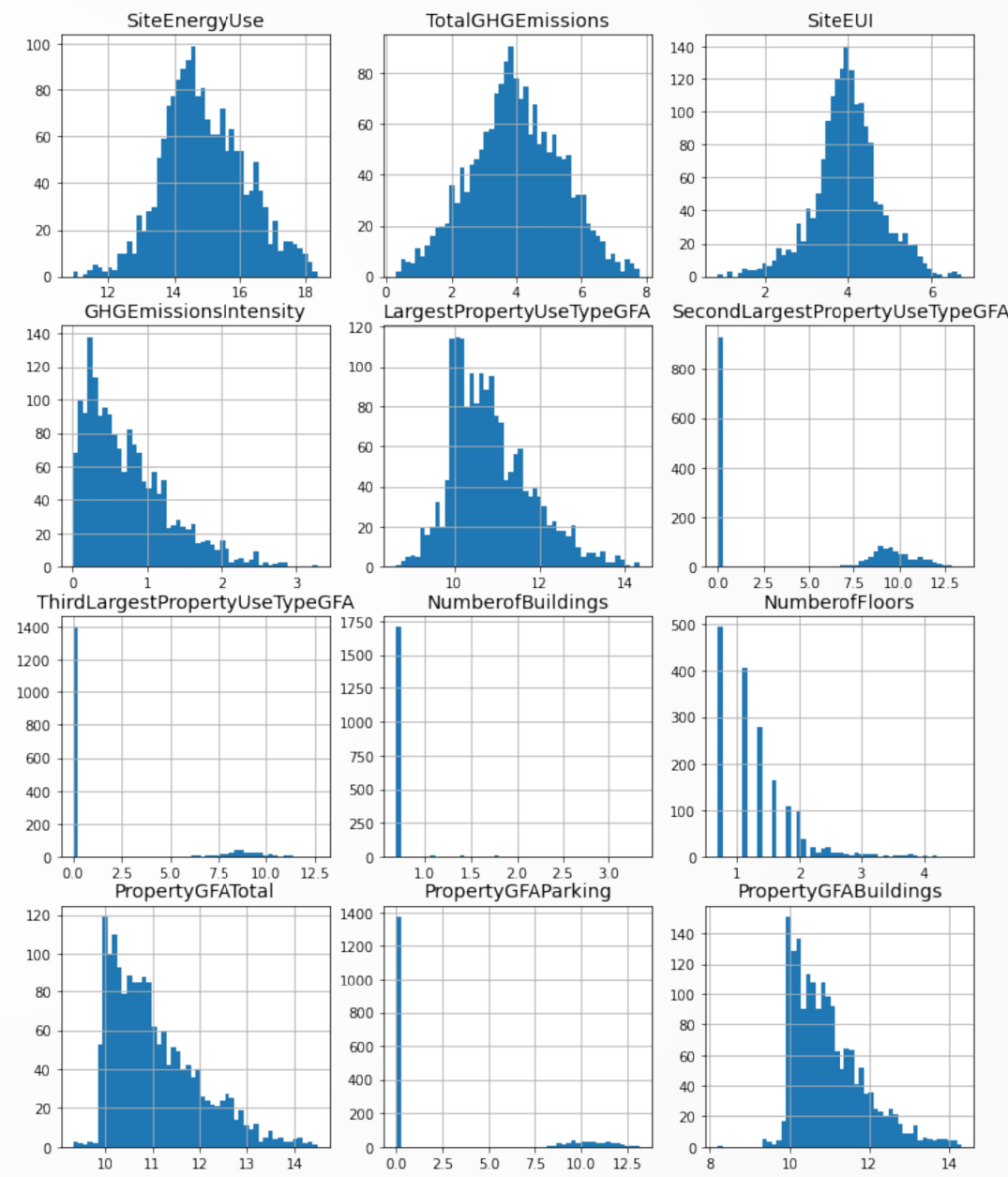
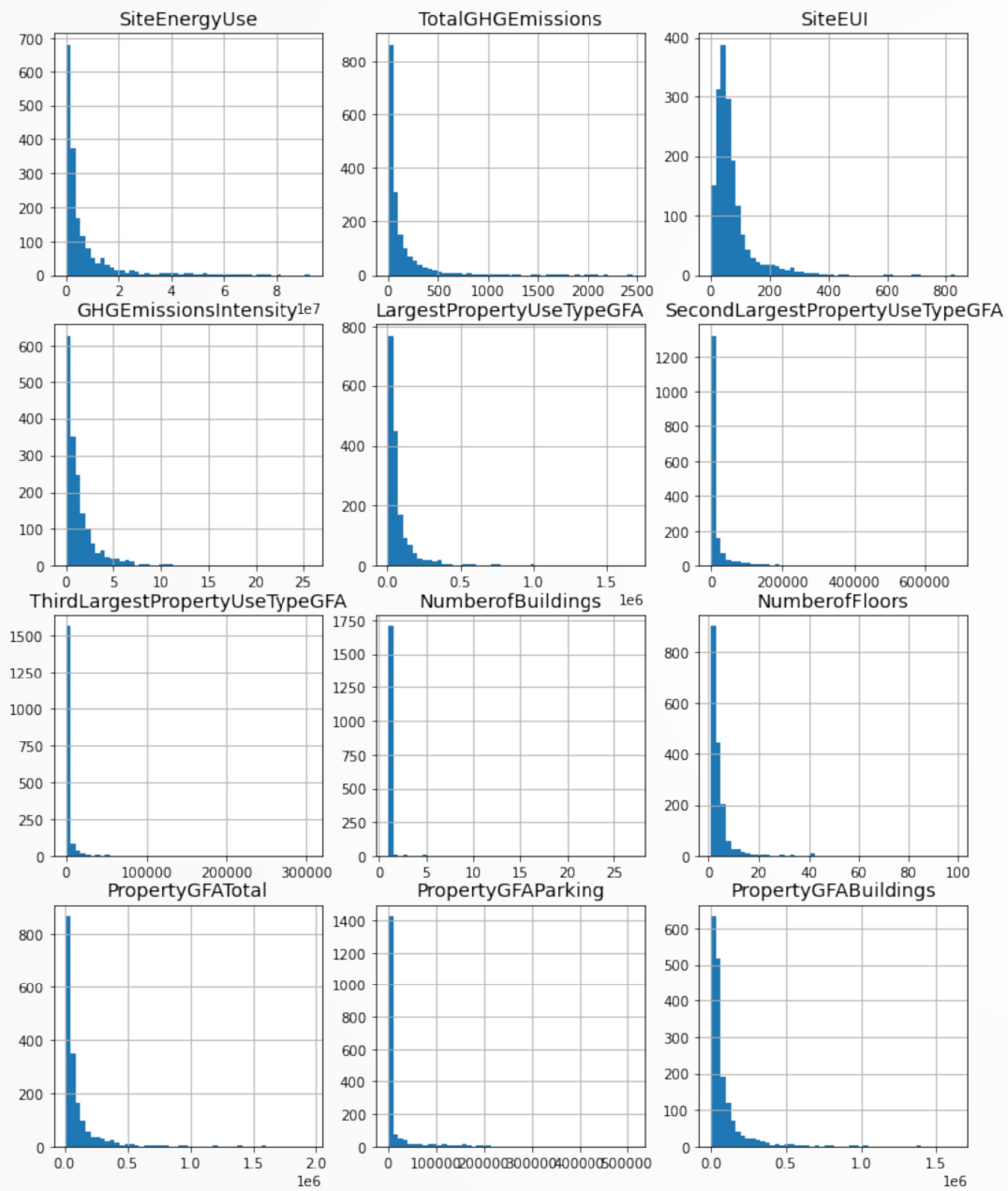


Transformation logarithmique de colonnes numériques



Feature engineering

- Variables transformés à logarithme
 - (Comparer la performance avec elles et sans)
- Distance de centre de Seattle
- ScoreTimesGFA (à comparer la performance avec et sans)

Modélisation

- La sélection de jeu de test sur le jeu de données entier, taille de teste 25 %
- La ligne de base d'erreurs : Dummy Regressor (le RMSE avec le moyen)
- Préparation de données avant de les passer au modèle : Column Transformer (centralisation et normalisation avec Standard Scaler pour les variables numériques, OneHotEncoder pour des variables catégorielles)
- Modélisation se fait en Pipeline avec la préparation de données et la recherche sur grille (GridSearchCV) sur le jeu d'entraînement, puis évaluation d'erreur avec le jeu de test
- Les métriques de l'évaluation d'erreurs : Root Mean Squared Error, Mean absolute Error, R2Score. Dans la présentation, Root Mean Squared Error sera présentée.

Modélisation : Linear Regression

- L'objectif de l'algorithme – maximiser la vraisemblance
- C'est la même chose que minimiser l'erreur quadratique moyen : la somme de carrés des distances entre les valeurs réelles et celles prédites
- Pas d'hyperparamètres à optimiser

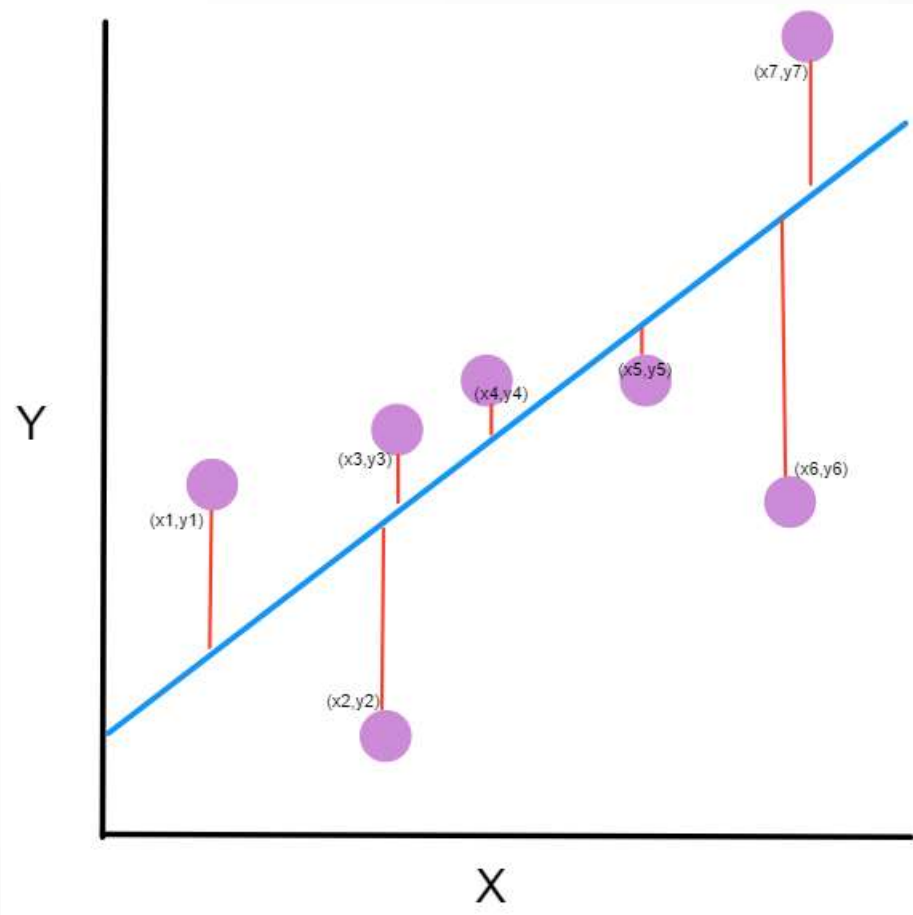
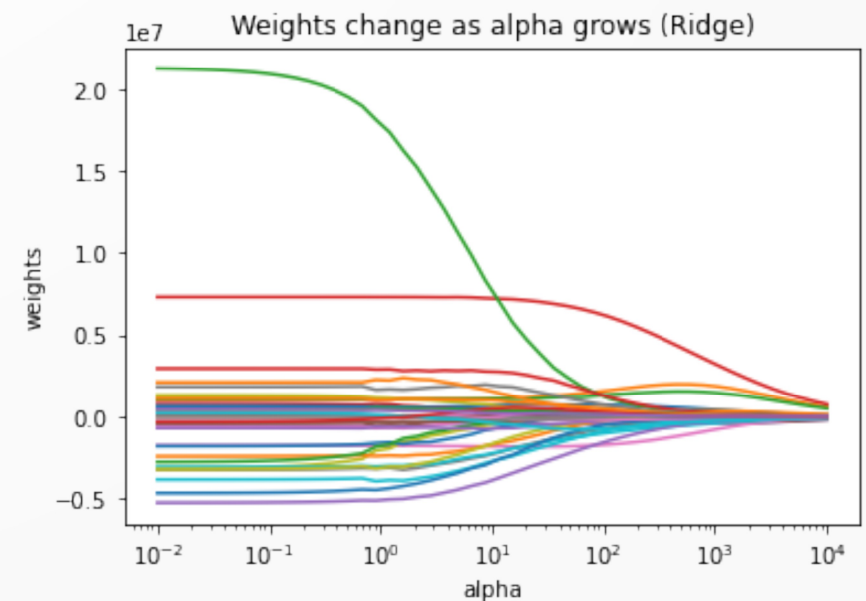
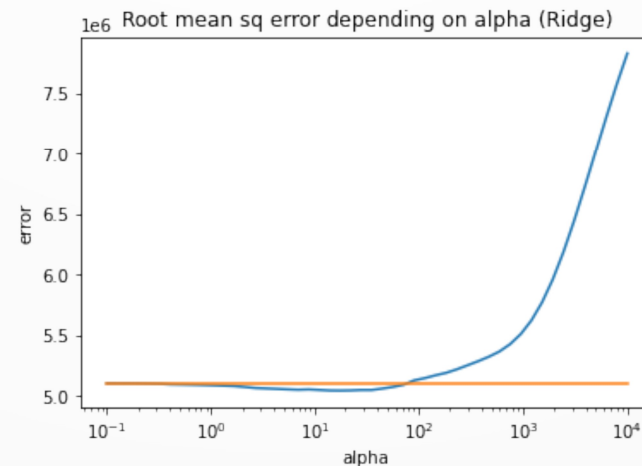
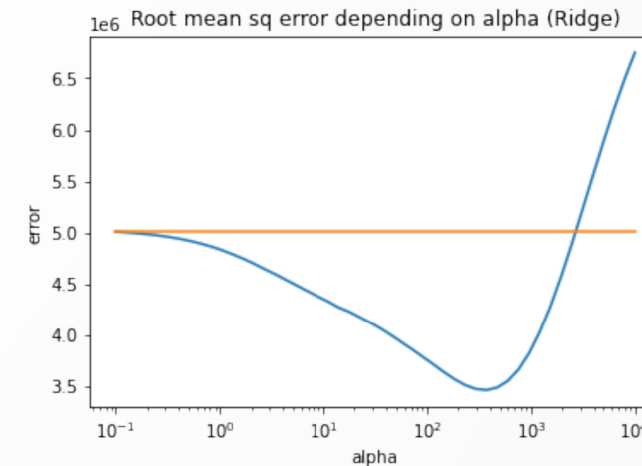
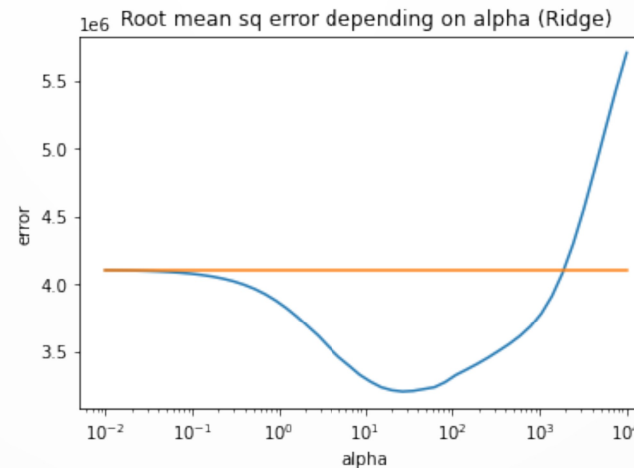


Image : free code camp

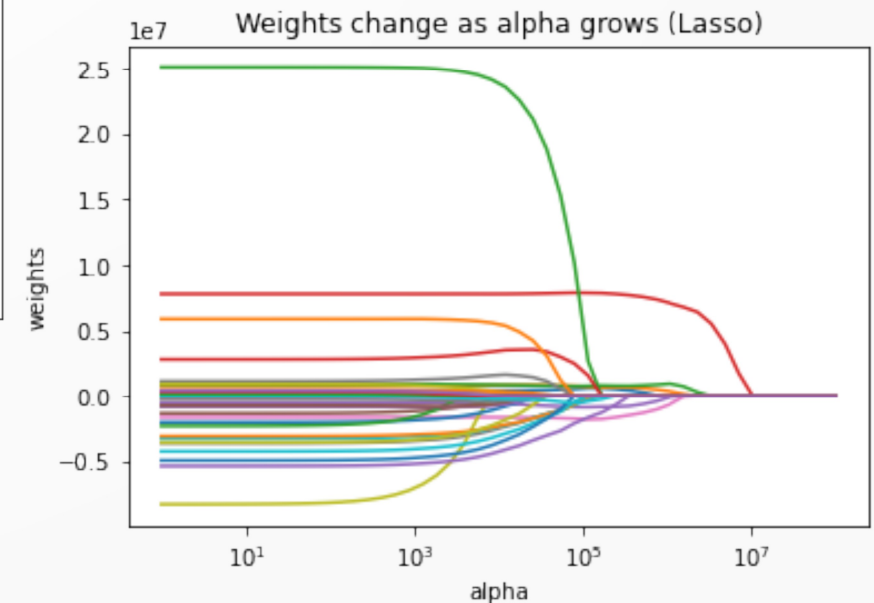
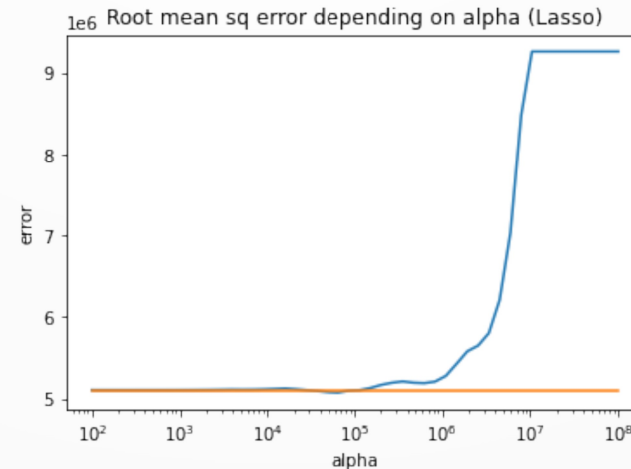
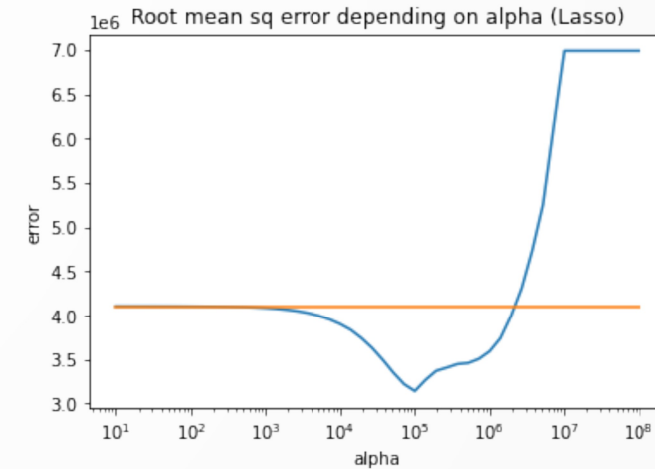
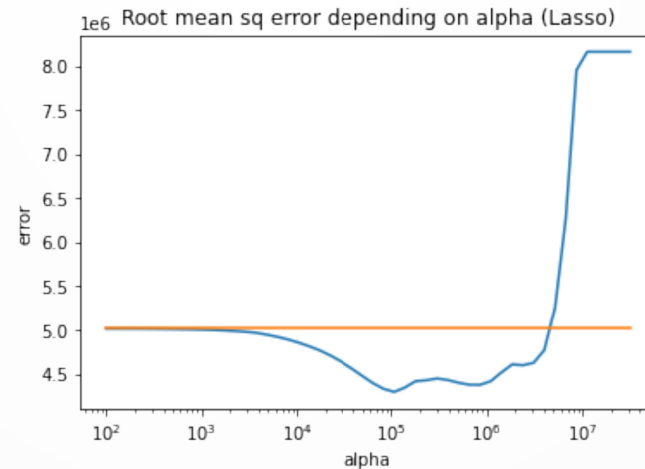
Modélisation : Ridge

- L'objectif de l'algorithme – minimiser l'erreur quadratique moyenne plus la norme 2 de poids de vecteurs de la régression au carré
- L'hyperparamètre à optimiser : α , coefficient de la norme de poids. Plus il est grand, plus la norme devient importante dans une fonction à minimiser, plus les poids de régression/composants de vecteurs sont petits.
- Les poids diminuent de manière équilibrée, sans disparaître (devenir zéro).



Modélisation : Lasso

- L'objectif de l'algorithme est à minimiser l'erreur quadratique moyenne plus la norme 1 de poids de vecteurs de la régression au module
- L'hyperparamètre à optimiser : α , coefficient de la norme de poids. Plus il est grand, plus la norme devient importante dans une fonction à minimiser, plus les poids de régression/composants de vecteurs sont petits.
- Les poids disparaissent l'un après l'autre, on l'appelle le principe de parcimonie : on a de moins en moins de variables coefficients non-zéro.



Modélisation : Elastic Net

- L'objectif de l'algorithme est à minimiser l'erreur quadratique moyenne plus la norme de poids de vecteurs de la régression.
- Quelle norme ici ? La norme à Elastic Net s'agit d'une combinaison de la norme 2 au carré et de la norme 1 au module. Ces deux normes sont combinées, chacune avec son coefficient :
- Pour la norme 1 il est $\alpha * l1_ratio$
- Pour la norme 2 il est $0.5 * \alpha * (1 - l1_ratio)$
- Les hyperparamètres à optimiser : $\alpha > 0$, $l1_ratio$ entre 0 et 1.

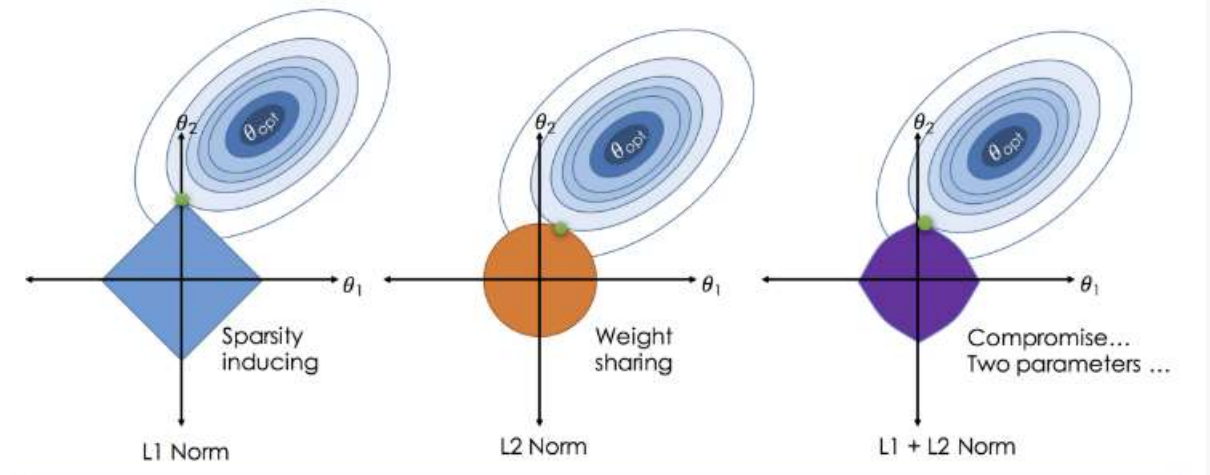
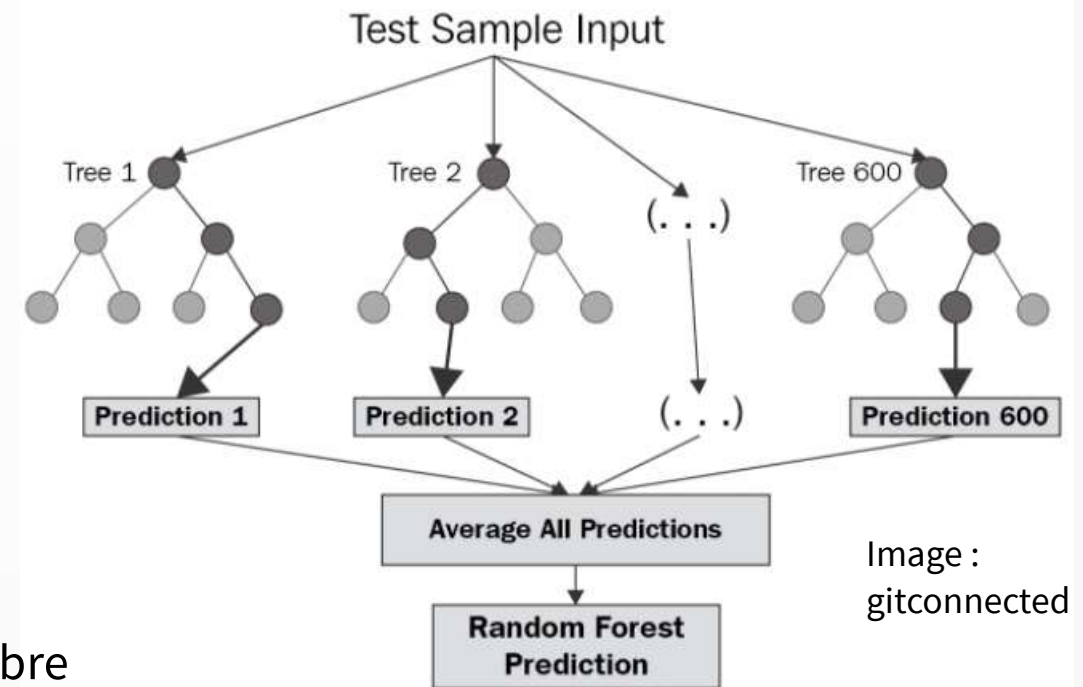


Image : free code camp

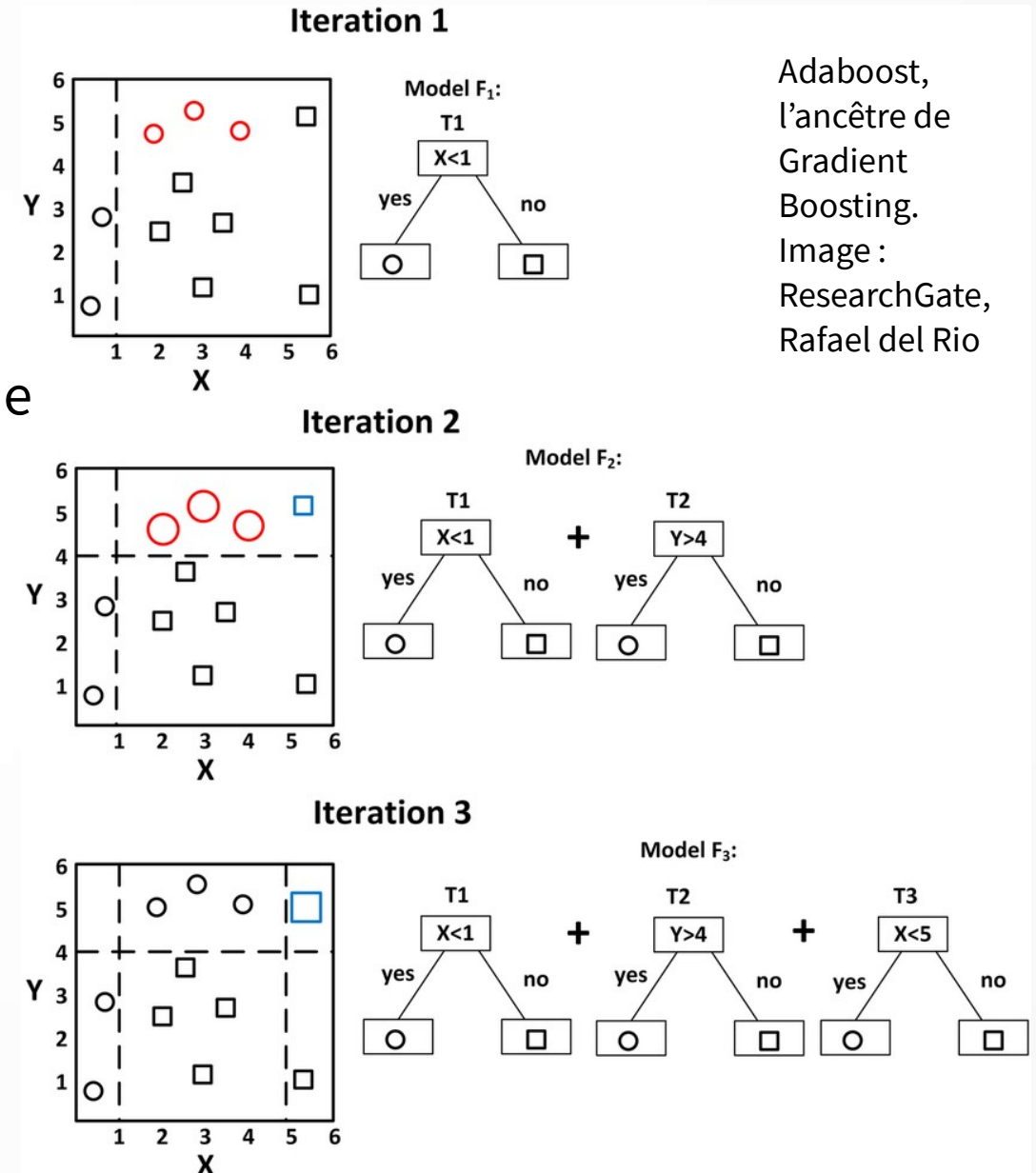
Modélisation : Random Forest

- Une forêt aléatoire est un ensemble d'arbres de décision. Chaque de ces arbres est construit sur un sous-échantillon aléatoire de l'ensemble de données. Chaque arbre est un apprenant faible – c'est-à-dire que ces prédictions ont de grandes erreurs. Puis, Random Forest rassemble toutes ces prédictions faibles et en prend le moyen. Cela permet d'améliorer la précision de la prédiction et éviter le sur-apprentissage.
- Les hyperparamètres à optimiser :
 - `n_estimators` : nombre d'arbres dans la forêt,
 - `max_samples` : la taille de sous-échantillon,
 - Les paramètres qui contrôlent la taille de l'arbre :
 - `max_depth` (la hauteur de l'arbre),
 - `min_samples_split` (combien d'individu il faut pour diviser)
 - `min_samples_leaf` (et combien il faut pour ne pas diviser)
 - `max_features` : combien en prendre pour construire un arbre
 - `min_impurity_decrease` : diviser une feuille seulement si ça améliore la variance de valeurs cible dans une feuille plus que ce seuil



Modélisation : Gradient Boosting

- Partant de valeur moyen de la colonne cible, il construit un prédicteur des résidus entre les valeurs observées et la prédiction précédente. Chaque itération on ajoute un arbre prédicteur de résidus, multiplié par la vitesse d'apprentissage. Les prédictions deviennent plus précises, donc les résidus à prédire deviennent plus petits : l'algorithme descend par le gradient.
- Les hyperparamètres à optimiser :
 - learning_rate
 - n_estimators
 - min_sample_split / min_sample_leaf
 - max_depth : la profondeur (taille) d'un arbre
 - max_leaf_nodes : le nombre maximale des feuilles, lié à max_depth
 - min_impurity_decrease



Adaboost,
l'ancêtre de
Gradient
Boosting.
Image :
ResearchGate,
Rafael del Rio

Modélisation : eXtreme Gradient Boosting

- L'algorithme construit les arbres XGBoost avec le score de similarité, à partir de valeur 0.5 toujours, pour des classifications ou régressions, mais on peut le définir nous-même aussi, et il décide ou mettre le seuil par le gain maximal de similarité, qu'on régularise avec **lambda**. Plus la sous-échantillon est petit, plus lambda pénalise le score de similarité et l'importance des valeurs de sortie. L'algorithme élague les feuilles dont le gain est moins que le seuil **gamma** : si le gain est moins que gamma, la feuille ne serait pas séparée.
- Les hyperparamètres à optimiser :
 - **n_estimators**, nombre des arbres
 - **Eta** : la vitesse d'apprentissage,
 - La taille d'un arbre :
 - Max_depth : la profondeur (taille) d'un arbre
 - Max_leaf_nodes : le nombre maximale des feuilles
 - **Gamma** : comme min_impurity_decrease dans Random Forest, mais sur le gain en similarité
 - Subsample et colsample_bytree : comme max_samples et max_features dans Random Forest
 - **Lambda** : norme L2, comme dans la régularisation Ridge
 - **Alpha** : norme L1, comme dans la régularisation Lasso

Modélisation : Support Vector Regressor

- Quand l'erreur est moins que l'épsilon donné, elle est considérée comme 0. Tout ce qui est plus haut, se compte en calcul d'erreur.
- Les hyperparamètres à optimiser :
 - Kernel : spécifie si l'on va appliquer une astuce de noyaux, et laquelle. L'astuce à noyaux c'est quand on remplace les variables par ces polynômes de certain degré (noyaux polynomial) ou de degré infini (noyaux exponentiel).
 - Des paramètres spécifiques au kernel
 - C : comme alpha dans Ridge, mais l'invers : plus il est grand, moins la régularisation est forte
 - Epsilon : la largeur de tube entre deux vecteurs de support

